

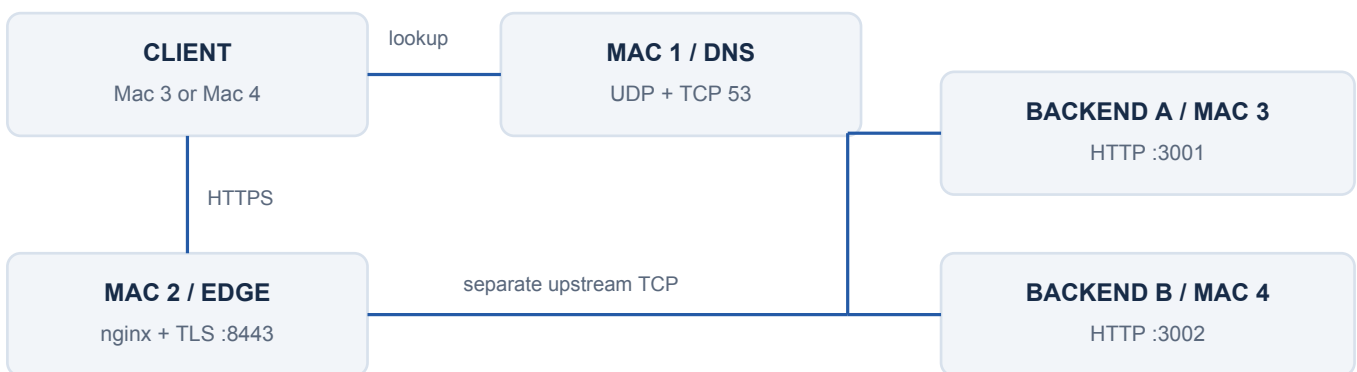
Computer Networks

Phase 1 implementation plan

Private Network Service Platform

A complete step-by-step guide for four Macs

20 steps. Complete backend code. DNS and nginx configurations. Trusted HTTPS. Packet analysis. Caching. Five controlled failures.



Scope: Tasks A-G and Section 6.3 failures. Phase 1: 50 marks, comprising 40 team marks and 10 individual viva marks.

This PDF contains instructions only. No project deployment, software installation, service startup, or system configuration is performed by creating it.

Replace all example IPs and the team1.test domain with measured values. Work from ~/cn-private-network and keep service terminals open. Combine all parts of any code block marked CONTINUED before running it.

Based on CN_Project_Doc.pdf, Sections 6, 9, and 10. Prepared 3 October 2026.

Step index

1. Assign the four machines	3
2. Install the required tools	3
3. Create the project folder on every Mac	4
4. Find each Mac's actual network details	5
5. Save addresses and prove LAN reachability	5
6. Create the backend application	6
7. Start backends and test them from Mac 2	8
8. Configure nginx over HTTP first	9
9. Configure and start private DNS	10
10. Configure two clients to use Mac 1 DNS	11
11. Generate the TLS certificate on Mac 2	12
12. Replace nginx's configuration with HTTPS	13
13. Trust certificates and verify HTTPS	14
14. Demonstrate HTTP caching	15
15. Capture the DNS-TCP-TLS journey	16
16. Demonstrate HTTP/2 if supported	17
17. Perform all five required failures	18
18. Prepare architecture and submission evidence	21
19. Rehearse the Phase 1 demonstration	22
20. Shut down and restore the clients	23
Reference notes	24

1. Assign the four machines

Machine	Role	Services
Mac 1	DNS server	dnsmasq, UDP/TCP 53
Mac 2	Edge and load balancer	nginx, HTTP 8080, HTTPS 8443
Mac 3	Backend A and test client	Python, TCP 3001
Mac 4	Backend B and test client	Python, TCP 3002

Mac 3 and Mac 4 will both use Mac 1 as their DNS resolver. The brief allows smaller teams to combine roles; adapt this four-Mac plan if you have fewer machines.

Connect all four Macs to the same private router, Wi-Fi, or controlled hotspot. Confirm that the network permits communication between devices. A router with DHCP reservations is helpful; do not assign arbitrary IPs without checking the network.

Keep the laptops plugged in and awake. If desired, run this in a separate terminal on each:

```
BASH
caffeinate -di
```

Leave it running during the demonstration and stop it with Ctrl+C afterward. It does not keep a closed laptop lid awake.

2. Install the required tools

If Homebrew is missing, follow its official installation and PATH instructions at <https://brew.sh>. Confirm brew --version in a new terminal.

On Mac 1:

```
BASH
brew install dnsmasq
```

On Mac 2:

```
BASH
brew install nginx openssl@3
export PATH="$(brew --prefix openssl@3)/bin:$PATH"
nginx -v
openssl version
```

Repeat the OpenSSL PATH export in any new Mac 2 terminal used to generate certificates.

On all Macs, check Python:

```
BASH
python3 --version
```

Use Python 3.9 or newer. If necessary:

BASH

```
brew install python
```

On Mac 3, install Wireshark:

BASH

```
brew install --cask wireshark
```

Mac 4 can install it too. Open Wireshark and confirm that it can capture the LAN interface. If capture permissions are missing, follow the installed application's ChmodBPF instructions. Test a short capture before the demo.

On Mac 3 and Mac 4:

BASH

```
curl --version  
dig -v
```

For this runbook, start nginx and dnsmasq using the project commands below rather than Homebrew background services. If another instance already runs, identify its owner and configuration before stopping it.

3. Create the project folder on every Mac

Use the same structure on all four machines:

BASH

```
mkdir -p ~/cn-private-network  
cd ~/cn-private-network  
  
mkdir -p backend dns nginx tls config runtime architecture captures demo  
mkdir -p evidence/{01-lan,02-dns,03-backends,04-load-balancing}  
mkdir -p evidence/{05-tls,06-caching,07-wireshark,08-failures,09-config}
```

Create .gitignore:

BASH

```
cat > .gitignore <<'EOF'  
.DS_Store  
**/*.key  
**/*.srl  
__pycache__/  
*.pyc  
runtime/  
config/network.env  
EOF
```

The public .crt file may be shared. The private .key file must remain on Mac 2. Do not force-add it to Git.

GitHub is optional; the assignment also permits a ZIP. If using GitHub, first create an empty repository through GitHub. Once source files are ready, run on the machine containing them:

BASH

```
git init -b main
git add .
git commit -m "Add Phase 1 networking project"
git remote add origin https://github.com/YOUR_USERNAME/cn-private-network.git
git push -u origin main
```

Replace the repository URL. Other members can clone instead of manually copying source files. Use separate branches or pull before editing shared files. Copy config/network.env separately because Git ignores it.

4. Find each Mac's actual network details

On every Mac:

BASH

```
networksetup -listallhardwareports
networksetup -listallnetworkservices
route -n get default
```

Identify the interface used by the private LAN. It might be en0, but verify it. Set the correct name:

BASH

```
IFACE="en0"
ipconfig getifaddr "$IFACE"
ifconfig "$IFACE"
```

Record IPv4, subnet mask/prefix, default gateway, active interface, MAC address, owner, and role. ifconfig may show a hexadecimal netmask; 0xffffffff corresponds to 255.255.255.0 or /24. Use the actual displayed mask.

Role	Example IPv4 - replace with actual
Mac 1 DNS	192.168.50.10
Mac 2 edge	192.168.50.11
Mac 3 backend A	192.168.50.12
Mac 4 backend B	192.168.50.13

Create architecture/network-inventory.md with your measured information. Include the network/SSID and a reachability matrix. These example addresses are not values to assign blindly.

5. Save addresses and prove LAN reachability

On every Mac, use your real addresses and assigned team domain in this file:

BASH

```
cd ~/cn-private-network

cat > config/network.env <<'EOF'
export DNS_IP="192.168.50.10"
```

BASH / CONTINUED

```
export EDGE_IP="192.168.50.11"
export BACKEND_A_IP="192.168.50.12"
export BACKEND_B_IP="192.168.50.13"
export TEAM_DOMAIN="team1.test"
export APP_HOST="app.team1.test"
export API_HOST="api.team1.test"
export APP_URL="https://app.team1.test:8443"
export API_URL="https://api.team1.test:8443"
EOF

source config/network.env
```

Repeat `source config/network.env` in every new testing terminal. Keep all domain variables consistent. Use `.test` rather than `.local`, which conflicts with macOS mDNS.

From each Mac, ping the other three machines:

BASH

```
ping -c 3 "$DNS_IP"
ping -c 3 "$EDGE_IP"
ping -c 3 "$BACKEND_A_IP"
ping -c 3 "$BACKEND_B_IP"
```

Your own address does not count as a peer-reachability test. Save the other three results for each machine under `evidence/01-lan`, labeled with the source Mac. For four Macs, document all twelve directed checks or a clear pairwise matrix.

Check relevant port conflicts on each role's Mac:

BASH

```
sudo lsof -nP -iUDP:53 -iTCP:53 -iTCP:8080 -iTCP:8443 \
-iTCP:3001 -iTCP:3002
```

Identify existing processes before stopping anything. Allow the project processes if macOS asks about incoming connections. If reachability fails, check IP/mask, same LAN, client isolation, and firewall policy; do not disable the entire firewall as a shortcut.

Checkpoint: every pair is reachable and the inventory is complete. Recheck addresses after changing networks.

6. Create the backend application

Create `backend/app.py` on Mac 3 and Mac 4 with identical code. The standard library is sufficient; no pip packages are needed.

BASH

```
cat > backend/app.py <<'PY'
import argparse
import hashlib
import json
from http.server import BaseHTTPRequestHandler, ThreadingHTTPServer
from urllib.parse import urlsplit

parser = argparse.ArgumentParser()
```

BASH / CONTINUED

```

parser.add_argument("--name", choices=("A", "B"), required=True)
parser.add_argument("--port", type=int, required=True)
args = parser.parse_args()

CACHE_BODY = json.dumps(
    {"message": "Phase 1 cache resource", "version": 1},
    sort_keys=True,
    separators=(",", ":"),
).encode()

ETAG = '"' + hashlib.sha256(CACHE_BODY).hexdigest() + '"'

class Handler(BaseHTTPRequestHandler):
    protocol_version = "HTTP/1.1"

    def respond(self, status, body=b"", cache=False):
        self.send_response(status)
        self.send_header("X-Backend", args.name)
        self.send_header(
            "Cache-Control",
            "public, max-age=60" if cache else "no-store",
        )

        if cache:
            self.send_header("ETag", ETAG)

        if status != 304:
            self.send_header("Content-Type", "application/json")
            self.send_header("Content-Length", str(len(body)))

        self.send_header("Connection", "close")
        self.end_headers()
        self.close_connection = True

        if self.command != "HEAD" and status != 304:
            self.wfile.write(body)

    def handle_request(self):
        path = urlsplit(self.path).path

        if path == "/cache":
            validators = self.headers.get("If-None-Match", "").split(",")
            matches = any(
                value.strip() == "*"
                or value.strip().removeprefix("W/") == ETAG
                for value in validators
            )

            self.respond(
                304 if matches else 200,
                CACHE_BODY,
                cache=True,
            )
            return

        if path in ("/", "/api/status"):
            body = json.dumps(
                {"backend": args.name, "status": "ok"}
            ).encode()
            self.respond(200, body)

```

BASH / CONTINUED

```

        return

        self.respond(404, b'{"error":"Not found"}')

    def do_GET(self):
        self.handle_request()

    def do_HEAD(self):
        self.handle_request()

server = ThreadingHTTPServer(("0.0.0.0", args.port), Handler)
print(
    f"Backend {args.name} listening on 0.0.0.0:{args.port}",
    flush=True,
)

try:
    server.serve_forever()
except KeyboardInterrupt:
    pass
finally:
    server.server_close()
PY

```

The /cache body deliberately stays identical on A and B. Identity remains visible in X-Backend. A shared strong ETag must correspond to identical representation data; this avoids invalid 304 responses when consecutive requests reach different backends.

7. Start backends and test them from Mac 2

Mac 3, service terminal:

BASH

```

cd ~/cn-private-network
python3 -u backend/app.py --name A --port 3001

```

Mac 4, service terminal:

BASH

```

cd ~/cn-private-network
python3 -u backend/app.py --name B --port 3002

```

Leave both terminals running. Use second terminals for testing; Ctrl+C stops that backend.

Mac 2:

BASH

```

cd ~/cn-private-network
source config/network.env

curl --noproxy '*' --connect-timeout 3 --max-time 10 \
  -i "http://$BACKEND_A_IP:3001/api/status" \
  | tee evidence/03-backends/backend-a.txt

```

BASH / CONTINUED

```
curl --noproxy '*' --connect-timeout 3 --max-time 10 \
-i "http://$BACKEND_B_IP:3002/api/status" \
| tee evidence/03-backends/backend-b.txt
```

Expected: HTTP 200, the correct X-Backend header, and JSON status ok. Also test / on both. Direct backend access is a setup check; the final application demonstration uses the edge domain.

Checkpoint: both backends respond independently from Mac 2. If either fails, check its terminal, LAN IP, 0.0.0.0 bind, listener, and incoming-connection permission.

8. Configure nginx over HTTP first

Mac 2:

BASH

```
cd ~/cn-private-network
source config/network.env

cat > nginx/nginx.conf <<EOF
worker_processes 1;
pid "$PWD/runtime/nginx.pid";
error_log "$PWD/runtime/nginx-error.log" info;

events {
    worker_connections 512;
}

http {
    log_format phasel '\$time_iso8601 client=\$remote_addr '
                    '"\$request" status=\$status '
                    'upstream=\$upstream_addr '
                    'upstream_status=\$upstream_status '
                    'backend=\$upstream_http_x_backend';

    access_log "$PWD/runtime/nginx-access.log" phasel;

    upstream backend_servers {
        server $BACKEND_A_IP:3001 max_fails=1 fail_timeout=5s;
        server $BACKEND_B_IP:3002 max_fails=1 fail_timeout=5s;
    }

    server {
        listen 8080;
        server_name $APP_HOST $API_HOST;

        location / {
            proxy_pass http://backend_servers;
            proxy_http_version 1.1;
            proxy_set_header Connection close;
            proxy_set_header Host \$host;
            proxy_set_header X-Forwarded-For \$proxy_add_x_forwarded_for;
            proxy_set_header X-Forwarded-Proto \$scheme;
            proxy_connect_timeout 2s;
            proxy_read_timeout 5s;
            proxy_next_upstream error timeout http_502 http_503 http_504;
            proxy_next_upstream_tries 2;
        }
    }
}
```

BASH / CONTINUED

```
    }
}
EOF
```

The backslashes before nginx variables preserve them while the shell substitutes your IPs and paths. Keep them exactly as shown in these heredoc commands.

Test and start:

BASH

```
nginx -p "$PWD/" -c "$PWD/nginx/nginx.conf" -t
nginx -p "$PWD/" -c "$PWD/nginx/nginx.conf"
```

Test repeated requests:

BASH

```
for i in 1 2 3 4 5 6; do
    curl --noproxy '*' -sS -D - -o /dev/null \
        http://127.0.0.1:8080/api/status
done | tee evidence/04-load-balancing/http-balancing.txt

tail -n 12 runtime/nginx-access.log
```

Expected: successful responses showing A and B. The default strategy is round-robin. A quiet sequential test is easiest to inspect; other clients/browser requests also consume turns. This configuration uses passive failure detection and retry, not continuous active health probes.

Checkpoint: HTTP through nginx reaches both backends. Always use the same prefix/config arguments for this nginx instance.

9. Configure and start private DNS

Mac 1:

BASH

```
cd ~/cn-private-network
source config/network.env

cat > dns/dnsmasq.conf <<EOF
port=53
listen-address=127.0.0.1,$DNS_IP
bind-interfaces
no-resolv
no-hosts
domain-needed
bogus-priv
local=/$TEAM_DOMAIN/
host-record=$APP_HOST,$API_HOST,$EDGE_IP,60
log-queries
log-facility=-
EOF
```

Both names point to Mac 2. The local rule keeps the project namespace local. This DNS-only configuration does not resolve unrelated internet names. Install tools before changing client DNS.

If internet DNS is needed, add a `server=ACTUAL_REACHABLE_UPSTREAM_DNS_IP` line, such as the router if it offers DNS. Do not point it back to Mac 1. No DHCP service is configured.

Test and start:

```
BASH
DNSMASQ_BIN="$(brew --prefix dnsmasq)/sbin/dnsmasq"

"$DNSMASQ_BIN" --test --conf-file="$PWD/dns/dnsmasq.conf"

sudo "$DNSMASQ_BIN" \
  --keep-in-foreground \
  --conf-file="$PWD/dns/dnsmasq.conf"
```

Keep this terminal open. `sudo` is needed for port 53. Query logs appear here.

From Mac 3 and Mac 4:

```
BASH
source config/network.env
dig "@$DNS_IP" "$APP_HOST" A
dig "@$DNS_IP" "$API_HOST" A
dig +tcp "@$DNS_IP" "$APP_HOST" A
```

Expected: both names return `EDGE_IP`; the `SERVER` line identifies `DNS_IP` and port 53. Ordinary queries generally use UDP. The extra TCP query checks TCP 53 support.

10. Configure two clients to use Mac 1 DNS

Perform this on both Mac 3 and Mac 4. Identify the actual network-service name:

```
BASH
networksetup -listallnetworkservices
```

Set it, replacing Wi-Fi if necessary. Save the original settings only once before changing them:

```
BASH
NET_SERVICE="Wi-Fi"

networksetup -getdnsservers "$NET_SERVICE" \
  > runtime/dns-before.txt

cat runtime/dns-before.txt

sudo networksetup -setdnsservers "$NET_SERVICE" "$DNS_IP"
sudo dscacheutil -flushcache
sudo killall -HUP mDNSResponder
```

Retain `dns-before.txt` for restoration. Do not overwrite it later with the project DNS settings. In new terminals, set `NET_SERVICE` to the same actual service name.

Verify:

BASH

```
networksetup -getdnsservers "$NET_SERVICE"

dig "$APP_HOST" A | tee evidence/02-dns/app-resolution.txt
dig "$API_HOST" A | tee evidence/02-dns/api-resolution.txt

curl --noproxy '*' --connect-timeout 3 --max-time 10 \
  -i "http://$APP_HOST:8080/api/status" \
  | tee evidence/04-load-balancing/http-domain.txt
```

The dig SERVER line should identify Mac 1. If dig succeeds but the app lookup fails, inspect scutil --dns and VPN/proxy/browser secure-DNS settings. Do not substitute hosts entries or curl --resolve for the final DNS demonstration.

Checkpoint: ordinary domain requests work from both configured clients. Label their evidence separately when combining files.

11. Generate the TLS certificate on Mac 2

The brief permits an OpenSSL self-signed certificate. Its separate local-CA alternative requires faculty approval. This plan uses the self-signed route with explicit client trust; server.crt itself is the demonstration trust anchor, not a separate CA.

Mac 2:

BASH

```
cd ~/cn-private-network
source config/network.env
export PATH="$(brew --prefix openssl@3)/bin:$PATH"

cat > tls/server.cnf <<EOF
[req]
prompt = no
distinguished_name = dn
x509_extensions = extensions

[dn]
CN = $APP_HOST
O = CN Phase 1 Team

[extensions]
subjectAltName = @names
basicConstraints = critical,CA:FALSE
keyUsage = critical,digitalSignature,keyEncipherment
extendedKeyUsage = serverAuth

[names]
DNS.1 = $APP_HOST
DNS.2 = $API_HOST
EOF
```

Generate once. Do not rerun over an existing certificate/key unless deliberately replacing them and updating every client's trust:

BASH

```
umask 077

openssl req -x509 -newkey rsa:2048 -sha256 -noenc \
  -days 30 \
  -keyout tls/server.key \
  -out tls/server.crt \
  -config tls/server.cnf

chmod 600 tls/server.key
chmod 644 tls/server.crt

openssl x509 -in tls/server.crt \
  -noout -subject -issuer -dates -ext subjectAltName

openssl verify -CAfile tls/server.crt tls/server.crt
```

Keep server.key on Mac 2. Transfer only server.crt to Macs 3/4, for example through AirDrop. If review is more than 30 days away, generate a fresh certificate before it and redistribute/retrust it. A domain change also needs a new matching certificate.

12. Replace nginx's configuration with HTTPS

Mac 2, in the project root with network.env loaded:

BASH

```
cat > nginx/nginx.conf <<EOF
worker_processes 1;
pid "$PWD/runtime/nginx.pid";
error_log "$PWD/runtime/nginx-error.log" info;

events {
    worker_connections 512;
}

http {
    log_format phasel1 '\$time_iso8601 client=\$remote_addr '
        '"\$request" status=\$status '
        'upstream=\$upstream_addr '
        'upstream_status=\$upstream_status '
        'backend=\$upstream_http_x_backend';

    access_log "$PWD/runtime/nginx-access.log" phasel1;
    gzip off;

    upstream backend_servers {
        server $BACKEND_A_IP:3001 max_fails=1 fail_timeout=5s;
        server $BACKEND_B_IP:3002 max_fails=1 fail_timeout=5s;
    }

    server {
        listen 8080;
        server_name $APP_HOST $API_HOST;
        return 301 https://\$host:8443\$request_uri;
    }

    server {
        listen 8443 ssl;
        server_name $APP_HOST $API_HOST;
```

BASH / CONTINUED

```

ssl_certificate "$PWD/tls/server.crt";
ssl_certificate_key "$PWD/tls/server.key";
ssl_protocols TLSv1.2 TLSv1.3;
ssl_session_cache off;
ssl_session_tickets off;

location = /edge-health {
    default_type application/json;
    add_header Cache-Control no-store;
    return 200 '{"edge":"ok"}';
}

location / {
    proxy_pass http://backend_servers;
    proxy_http_version 1.1;
    proxy_set_header Connection close;
    proxy_set_header Host $host;
    proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    proxy_set_header X-Forwarded-Proto $scheme;
    proxy_connect_timeout 2s;
    proxy_read_timeout 5s;
    proxy_next_upstream error timeout http_502 http_503 http_504;
    proxy_next_upstream_tries 2;
}
}
EOF

nginx -p "$PWD/" -c "$PWD/nginx/nginx.conf" -t
nginx -p "$PWD/" -c "$PWD/nginx/nginx.conf" -s reload

```

The running edge now accepts TLS on 8443 and redirects HTTP 8080. Session caching/tickets are off to make fresh handshake evidence straightforward. nginx needs no sudo on these unprivileged ports.

13. Trust certificates and verify HTTPS

On both Mac 3 and Mac 4:

BASH

```

cd ~/cn-private-network
cp ~/Downloads/server.crt tls/server.crt

openssl x509 -in tls/server.crt -noout -fingerprint -sha256

```

Adjust the downloaded filename if AirDrop renamed it. Compare fingerprints against Mac 2; they must match.

Install explicit SSL trust:

BASH

```

sudo security add-trusted-cert \
    -d \
    -r trustRoot \
    -p ssl \
    -k /Library/Keychains/System.keychain \
    "$PWD/tls/server.crt"

```

Test with explicit certificate validation:

BASH

```
source config/network.env

curl --noproxy '*' --cacert tls/server.crt \
  --connect-timeout 3 --max-time 12 \
  -v -i "$APP_URL/api/status" \
  2>&1 | tee evidence/05-tls/https-validation.txt

curl --noproxy '*' --cacert tls/server.crt \
  -i "$API_URL/api/status"
```

--cacert validates certificate/hostname; it does not bypass validation. It works with curl builds that use a different trust store from the browser. The browser separately uses the installed trust.

Open the actual https://app.YOUR_TEAM.test:8443/ URL in a browser on each client. There must be no certificate warning. Do not click through one. Check fingerprint, SAN, dates, client clock, and trust if there is a warning; restart the browser after adding trust if needed. Save the successful browser response and certificate/security view.

Show HTTPS balancing:

BASH

```
for i in 1 2 3 4 5 6; do
  curl --noproxy '*' --cacert tls/server.crt \
    --http1.1 --connect-timeout 3 --max-time 12 \
    -sS -D - -o /dev/null "$APP_URL/api/status"
done | tee evidence/04-load-balancing/https-balancing.txt
```

Checkpoint: browser trust works, validating curl succeeds without -k, and both backends appear. Use the non-cacheable status endpoint for balancing/failure evidence.

14. Demonstrate HTTP caching

From a configured client:

BASH

```
curl --noproxy '*' --cacert tls/server.crt \
  -sS \
  -D evidence/06-caching/initial.headers \
  -o evidence/06-caching/initial.json \
  "$APP_URL/cache"

cat evidence/06-caching/initial.headers
cat evidence/06-caching/initial.json

curl --noproxy '*' --cacert tls/server.crt \
  -I "$APP_URL/cache" \
  | tee evidence/06-caching/head.txt
```

Expected: 200, Cache-Control: public, max-age=60, and a quoted ETag. Extract the actual ETag and send a conditional GET:

BASH

```
ETAG="$({
  awk 'tolower($1)=="etag:" {gsub("\r", "", $2); print $2; exit}' \
    evidence/06-caching/initial.headers
})"

curl --noproxy '*' --cacert tls/server.crt \
  -sS \
  -H "If-None-Match: $ETAG" \
  -D evidence/06-caching/conditional.headers \
  -o evidence/06-caching/conditional.body \
  "$APP_URL/cache"

cat evidence/06-caching/conditional.headers
wc -c evidence/06-caching/conditional.body
```

Expected: 304 Not Modified and zero body bytes. Test a nonmatching validator:

BASH

```
curl --noproxy '*' --cacert tls/server.crt \
  -i \
  -H 'If-None-Match: "different-version"' \
  "$APP_URL/cache" \
  | tee evidence/06-caching/nonmatching.txt
```

Expected: 200 with the body. nginx proxy caching is not enabled; this demonstrates origin cache headers and conditional validation, which the brief accepts.

Case	Behavior
Fresh client cache hit	Cached content can be reused without a request
Revalidation	Conditional request; unchanged content can produce 304
Full response	200 with the representation body

Ordinary curl does not automatically demonstrate browser cache reuse. Explain the distinction in the viva.

15. Capture the DNS-TCP-TLS journey

Use Mac 3 or Mac 4 as the capture client. Mac 1 DNS and Mac 2 edge must both be remote. In Wireshark, select the actual LAN interface and start recording before making the request. An optional capture filter is:

TEXT

```
port 53 or tcp port 8443
```

In the client terminal:

BASH

```
sudo dscacheutil -flushcache
sudo killall -HUP mDNSResponder
```

BASH / CONTINUED

```
curl --noproxy '*' --cacert tls/server.crt \
  --http1.1 \
  --tlsv1.2 --tls-max 1.2 \
  --connect-timeout 3 --max-time 12 \
  -v "$APP_URL/api/status" \
  2>&1 | tee evidence/07-wireshark/flow-curl.txt
```

Stop and save captures/phase1-flow.pcapng. The TLS 1.2 constraint applies only to this evidence request. It makes Certificate and ChangeCipherSpec visible for the brief while application data remains encrypted. TLS 1.3 encrypts handshake messages after ServerHello, including Certificate.

Useful display filters after recording:

TEXT

```
dns
tcp.port == 8443
tls
tcp.flags.syn == 1
tls.handshake.type == 1 || tls.handshake.type == 2 || tls.handshake.type == 11
```

Use one filter at a time as needed. If 8443 is not decoded as TLS, use Analyze > Decode As for that TCP stream/port.

Evidence	Identify
DNS	Name, answer containing edge IP, transaction ID, client/DNS IPs, ports
TCP	SYN/SYN-ACK/ACK, ephemeral client port, destination 8443
Reliability	Sequence and acknowledgement numbers; receive window concept
TLS	ClientHello, ServerHello, Certificate, ChangeCipherSpec
Encryption	TLS application data
HTTP	Request/response headers from matching curl output

Save readable expanded packet screenshots and actual frame numbers. The third ACK may carry TLS data; handshake messages can share or span packets. Select the correct tcp.stream rather than unrelated traffic.

Do not run a separate dig and claim that its packets are curl's actual lookup. Confirm the main capture contains the request's DNS query; if absent, clear caches and repeat with a fresh capture/process.

Explain that nginx creates a separate TCP connection to a backend. Client-to-edge is HTTPS; edge-to-backend is HTTP. Optional upstream evidence should be captured on Mac 2: a client normally cannot capture traffic between two other Macs on a switched LAN.

16. Demonstrate HTTP/2 if supported

Mac 2:

BASH

```
nginx -V 2>&1
```

Check for `--with-http_v2_module`. For nginx 1.25.1 or newer, add this inside the HTTPS server block:

NGINX

```
http2 on;
```

Test and reload:

BASH

```
nginx -p "$PWD/" -c "$PWD/nginx/nginx.conf" -t
nginx -p "$PWD/" -c "$PWD/nginx/nginx.conf" -s reload
```

On a client whose `curl --version` lists HTTP2:

BASH

```
curl --noproxy '*' --cacert tls/server.crt \
  --http2 --connect-timeout 3 --max-time 12 \
  -i "$APP_URL/api/status"
```

Confirm the response actually reports HTTP/2. A browser's Network protocol column is another option. If nginx lacks support, document that limitation. Older nginx uses different configuration syntax; keep the baseline unless adapting it deliberately. The edge can speak HTTP/2 while upstream remains HTTP/1.1. HTTP/3 and email protocols are explanation-only for this project.

17. Perform all five required failures

Perform one failure at a time. Restore and verify the healthy system before the next. Record actual output rather than treating expected results as observed evidence.

A. Wrong DNS server

On one configured client, confirm `NET_SERVICE` identifies the active service. Point DNS at Mac 2, where nothing should listen on 53:

BASH

```
sudo networksetup -setdnsservers "$NET_SERVICE" "$EDGE_IP"
sudo dscacheutil -flushcache
sudo killall -HUP mDNSResponder

dig +time=2 +tries=1 "$APP_HOST" A \
  > evidence/08-failures/wrong-dns.txt 2>&1

ping -c 3 "$EDGE_IP" \
  > evidence/08-failures/wrong-dns-ping.txt
```

Expected: lookup fails while IP reachability remains. Restore:

BASH

```
sudo networksetup -setdnsservers "$NET_SERVICE" "$DNS_IP"
sudo dscacheutil -flushcache
sudo killall -HUP mDNSResponder
dig "$APP_HOST" A
```

B. Wrong DNS record

On Mac 1, stop only the project dnsmasq with Ctrl+C. In its project root, with network.env loaded:

BASH

```
cp dns/dnsmasq.conf runtime/dnsmasq.good.conf

sed "s/$EDGE_IP/$BACKEND_A_IP/g" \
  runtime/dnsmasq.good.conf > dns/dnsmasq.conf

sudo "$DNSSMASQ_BIN" \
  --keep-in-foreground \
  --conf-file="$PWD/dns/dnsmasq.conf"
```

Use Mac 4 as client. Confirm Mac 3 has no listener on 8443; flush DNS and test:

BASH

```
sudo dscacheutil -flushcache
sudo killall -HUP mDNSResponder

dig "$APP_HOST" A \
  | tee evidence/08-failures/wrong-record-dns.txt

curl --noproxy '*' --cacert tls/server.crt \
  --connect-timeout 3 --max-time 10 \
  -v "$APP_URL/api/status" \
  > evidence/08-failures/wrong-record-request.txt 2>&1
```

Expected: DNS succeeds with Mac 3's address; connecting to its unused 8443 fails. Restore on Mac 1: stop dnsmasq and run:

BASH

```
cp runtime/dnsmasq.good.conf dns/dnsmasq.conf

sudo "$DNSSMASQ_BIN" \
  --keep-in-foreground \
  --conf-file="$PWD/dns/dnsmasq.conf"
```

Flush the client cache and verify normal HTTPS again. This is a wrong-record failure; TTL cutover behavior is Phase 2.

C. Backend A stopped

On Mac 3, Ctrl+C Backend A. On a client:

BASH

```
for i in 1 2 3 4 5 6; do
    curl --noproxy '*' --cacert tls/server.crt \
        --connect-timeout 3 --max-time 12 \
        -sS -D - -o /dev/null "$APP_URL/api/status"
done | tee evidence/08-failures/backend-a-down.txt
```

Expected: successful B responses. The first attempted A connection can fail internally and be retried to B. Save matching nginx access/error logs on Mac 2.

Restart on Mac 3:

BASH

```
python3 -u backend/app.py --name A --port 3001
```

Allow the five-second fail_timeout to pass; then make repeated requests and save A/B recovery evidence. Detection and recovery are based on real traffic.

D. Both backends stopped

Stop both Python servers. From a client:

BASH

```
curl --noproxy '*' --cacert tls/server.crt \
    --connect-timeout 3 --max-time 12 \
    -v -i "$APP_URL/api/status" \
    > evidence/08-failures/both-backends-down.txt 2>&1

curl --noproxy '*' --cacert tls/server.crt \
    -i "$APP_URL/edge-health" \
    | tee evidence/08-failures/edge-still-running.txt
```

Expected: normally 502 for /api/status and 200 for nginx's /edge-health; TLS validation still succeeds. Explain that the fault is upstream of the edge. Restart both and verify recovery after fail_timeout.

E. Wrong destination port

On Mac 2, confirm no listener on 9443:

BASH

```
lsof -nP -iTCP:9443 -sTCP:LISTEN
```

From a client:

BASH

```
dig "$APP_HOST" A \
    | tee evidence/08-failures/wrong-port-dns.txt

curl --noproxy '*' --cacert tls/server.crt \
    --connect-timeout 3 --max-time 10 \
    -v "https://$APP_HOST:9443/api/status" \
    > evidence/08-failures/wrong-port.txt 2>&1

curl --noproxy '*' --cacert tls/server.crt \
```

BASH / CONTINUED

```
-i "$APP_URL/api/status"
```

Expected: lookup succeeds, the unused port refuses or times out, and 8443 works. Record the actual error. No configuration restoration is needed for this test.

18. Prepare architecture and submission evidence

Include topology with actual IPs/roles; complete inventory; protocol request-flow diagram; service ports; TLS termination; and the separate upstream connection. Explain the local roles' cloud equivalents without deploying to cloud.

TEXT**Client**

```
-> DNS query to Mac 1
<- Mac 2 address
-> TCP handshake with Mac 2:8443
-> TLS handshake and certificate validation
-> Encrypted HTTP request to nginx
-> Separate HTTP connection to Backend A or B
<- Backend response
<- Encrypted response to client
```

Folder	Required contents
01-lan	Inventory and all-pair ping results
02-dns	Both clients' settings and both domain answers
03-backends	Direct A/B checks from Mac 2
04-load-balancing	Domain-based HTTP/HTTPS A/B responses
05-tls	Certificate details, validating curl, browser evidence
06-caching	Initial 200, matching 304, nonmatching 200
07-wireshark	Screenshots, frame numbers, matching curl output
08-failures	All five failures and recovery evidence
09-config	Actual configs and launch/certificate notes

Keep Mac 3 and Mac 4 evidence separately labeled when combining folders. Retain raw outputs alongside screenshots and .pcapng files.

On Mac 1, after restoring the correct record:

BASH

```
cp dns/dnsmasq.conf evidence/09-config/dnsmasq.conf
```

On Mac 2:

BASH

```
cp nginx/nginx.conf evidence/09-config/nginx.conf
cp tls/server.cnf evidence/09-config/server.cnf
cp tls/server.crt evidence/09-config/server.crt
```

Do not copy server.key. Include complete backend source and launch instructions. This plan can serve as certificate setup notes. Create an evidence index with filenames/frame numbers so every item is findable within 30 seconds. Check staged files before pushing or creating the submission ZIP.

19. Rehearse the Phase 1 demonstration

Before a fresh start, verify IPs and certificate validity. If addresses changed, update network.env on all Macs and regenerate the DNS/nginx configs with the relevant commands. Start A on Mac 3, B on Mac 4, DNS on Mac 1, then nginx on Mac 2. For an already running edge, reload rather than start another instance.

Live sequence:

- Topology and inventory.
- LAN reachability.
- Domain resolution through Mac 1.
- HTTPS through the domain without warnings.
- Responses from A and B.
- Saved DNS/TCP/TLS packet evidence.
- Cache headers and 304.
- Backend failure and recovery.
- Other required failure evidence.
- Individual viva.

Each member should explain DNS versus connectivity, ports, TCP reliability, TLS trust, round-robin/passive failover, fresh cache versus revalidation, and layered diagnosis. No member should rely only on their own component.

Review 1 area	Marks
LAN + private DNS	10
Backends + reverse proxy + balancing	10
HTTPS/TLS correctness and explanation	8
Packet analysis and protocol flow	7
HTTP caching and transport understanding	5
Individual viva	10
Total	50

Section 8's Phase 2 demo items are deferred. Completion requires a working live system and actual evidence, not only configurations or expected output text.

20. Shut down and restore the clients

On Mac 2:

```
BASH
nginx -p "$PWD/" -c "$PWD/nginx/nginx.conf" -s quit
```

Stop dnsmasq and both backends with Ctrl+C in their service terminals. On each configured client, set the actual NET_SERVICE again if needed and inspect its saved original settings:

```
BASH
NET_SERVICE="Wi-Fi"
cat runtime/dns-before.txt
```

If the original output said no manual DNS servers were configured, restore automatic DNS:

```
BASH
sudo networksetup -setdnsservers "$NET_SERVICE" empty
```

If it listed specific servers, restore those exact addresses instead:

```
BASH
sudo networksetup -setdnsservers "$NET_SERVICE" ORIGINAL_DNS_1 ORIGINAL_DNS_2
```

Replace placeholders and include exactly the saved list. Then:

BASH

```
sudo dscacheutil -flushcache
sudo killall -HUP mDNSResponder
networksetup -getdnsservers "$NET_SERVICE"
```

Retain certificate trust while rehearsing. When permanently removing project trust:

BASH

```
sudo security remove-trusted-cert -d "$PWD/tls/server.crt"
```

If a certificate copy remains in Keychain Access, identify it by its fingerprint and remove only that project certificate. Stop caffeinate with Ctrl+C.

Reference notes

Primary assignment: CN_Project_Doc.pdf, Section 6 (Tasks A-G and failures), Section 9 (deliverables), and Section 10 (Review 1 rubric). The final running system is entirely local. No implementation or four-Mac runtime validation was performed to create this PDF.

Official technical references:

- Homebrew installation: <https://brew.sh>
- nginx round-robin and passive failures: https://nginx.org/en/docs/http/load_balancing.html
- nginx command-line options: <https://nginx.org/en/docs/switches.html>
- nginx proxy options: https://nginx.org/en/docs/http/nginx_http_proxy_module.html
- nginx HTTP/2 and version requirement: https://nginx.org/en/docs/http/nginx_http_v2_module.html
- dnsmasq configuration: <https://thekelleys.org.uk/dnsmasq/docs/dnsmasq-man.html>
- OpenSSL certificate generation: <https://docs.openssl.org/3.0/man1/openssl-req/>
- curl certificate verification: <https://curl.se/docs/sslcerts.html>
- curl TLS version limits: <https://curl.se/docs/manpage.html#--tls-max>
- HTTP validators and conditional responses: <https://www.rfc-editor.org/rfc/rfc9110.html>
- TLS 1.3 handshake encryption: <https://www.rfc-editor.org/rfc/rfc8446>